

人工智能课程复习大纲

课程: 人工智能 (Artificial Intelligence) 学校: 中山大学 (SYSU) · 计算机科学与工程学院 授课教师: 陈川 教授
教材: *Artificial Intelligence: A Modern Approach*, S. Russell & P. Norvig, 第3版 课程网站:
<https://easyhpc.net/course/143>

目录

1. [第一讲：人工智能引论](#)
2. [第二讲：知识表示与推理 I —— 逻辑基础](#)
3. [第三、四讲：知识表示与推理 II —— 归结原理](#)
4. [第五、六讲：搜索技术 I —— 盲目搜索与启发式搜索](#)
5. [第七讲：搜索技术 II —— 对抗搜索与博弈](#)
6. [第八讲：高级搜索技术 —— 局部搜索与优化](#)
7. [第九讲：概率推理与不确定性](#)
8. [第十讲：决策树](#)
9. [第十一讲：机器学习基础](#)
10. [第十二讲：无监督学习 —— 聚类](#)
11. [第十三讲：人工神经网络](#)
12. [第十四讲：深度学习 I —— CNN/RNN/LSTM](#)
13. [第十五讲：深度学习 II —— Transformer](#)
14. [第十六讲：强化学习](#)

第一讲：人工智能引论

1.1 课程信息

- 教材与参考书:
 - 主教材: Russell & Norvig, *AI: A Modern Approach*, 第3版
 - 辅助: Python + Keras + TensorFlow (2019)
- 成绩评定: 期末考试 40% + 平时作业 30% + 期中 20% + 出勤 10% (或其他组合方案)
- 课件助教: luoy585, songzh53 (@mail2.sysu.edu.cn)

1.2 什么是人工智能

- 图灵测试 (Turing Test, 1950): Alan Turing, "Computing Machinery and Intelligence" — "Can Machines think?"
- **AI** 的定义: 多视角定义 (思维/行动、人类理性标准)
- 达特茅斯会议 (Dartmouth Workshop, 1956): AI 正式诞生, John McCarthy 提出"人工智能"一词

1.3 AI 发展历史

时期	事件
1950s	图灵测试、逻辑理论家、达特茅斯会议
1970s	专家系统兴起
1980s	知识工程、连接主义萌芽
1987-1993	第二次 AI 寒冬
1997	IBM 深蓝击败国际象棋冠军 Kasparov
2011	IBM Watson 在 Jeopardy! 获胜
2012	ImageNet + AlexNet, 深度学习兴起
2016	AlphaGo 击败李世石
2017	AlphaGo Zero（零人类知识）、Transformer 架构
2022	ChatGPT 发布，大语言模型 (LLM) 时代

1.4 AI 三大学派

学派	英文	代表人物	代表系统
符号主义	Symbolicism	John McCarthy	Deep Blue, Watson, 专家系统
连接主义	Connectionism	Hinton, LeCun, Bengio	MLP, CNN, ImageNet, AlphaGo
行为主义	Actionism/Behaviorism	Brooks	Boston Dynamics 机器人

1.5 重要里程碑与应用

- **AlphaGo / AlphaGo Zero / AlphaZero:** DeepMind，围棋/国际象棋/将棋
- **DeepSeek:** 国产大语言模型，性能接近 GPT-4，成本仅 1/10~1/18
- **Style2paints:** AI 动漫上色
- **CycleGAN:** 不成对图像翻译
- **NeRF:** 神经辐射场 3D 重建
- **AlphaStar:** StarCraft II 99.8% 胜率

1.6 AI 相关图灵奖得主

年份	获奖者	贡献
1969	Marvin Minsky	AI 框架理论
1971	John McCarthy	AI 奠基、LISP 语言
1975	Simon & Newell	通用问题求解器
1994	Feigenbaum & Reddy	专家系统
2010	Leslie Valiant	PAC 学习理论 (1984)
2011	Judea Pearl	概率推理、贝叶斯网络、因果推理
2018	Bengio, Hinton, LeCun	深度学习

1.7 课程核心主题预览

1. 智能 Agent
2. 搜索技术（盲目搜索、启发式搜索、对抗搜索）
3. 知识表示与逻辑推理（归结原理）
4. 概率推理与决策
5. 机器学习（监督/无监督/半监督/强化学习）
6. 决策树
7. 人工神经网络与深度学习
8. Transformer, GAN 等前沿架构

第二讲：知识表示与推理 I —— 逻辑基础

2.1 课程定位

- 本讲是知识表示与推理的第一部分（命题逻辑 + 一阶谓词逻辑基础）
- 第三、四讲是本讲的延续（归结原理与自动定理证明）
- 对应教材第7-8章

2.2 知识与知识表示

知识的类型

- 陈述性知识、过程性知识
- 知识的层次：数据 → 信息 → 知识 → 智慧

知识表示方法

- 逻辑表示法
- 产生式规则
- 语义网络
- 框架表示法

- 神经网络表示法

2.3 命题逻辑 (Propositional Logic) ★

基本概念

- 命题 (Proposition): 具有确定真值的陈述句 (True/False)
- 原子命题: 不可再分的命题
- 复合命题: 由原子命题通过联结词组合

语法 (Syntax)

- 命题常量 True(T) 和 False(F) 是公式
- 命题变元 P, Q, R, ... 是公式
- 若 A 是公式, 则 $(\neg A)$ 是公式
- 若 A, B 是公式, 则 $(A \wedge B)$, $(A \vee B)$, $(A \rightarrow B)$, $(A \leftrightarrow B)$ 是公式
- 有限次应用以上规则得到的都是公式

逻辑联结词

联结词	符号	含义
否定	$\neg$	NOT
合取	$\wedge$	AND
析取	$\vee$	OR
蕴含	$\rightarrow$	IF...THEN (等价于 $\neg P \vee Q$)
双条件	$\leftrightarrow$	当且仅当 $(P \rightarrow Q) \wedge (Q \rightarrow P)$

语义 (Semantics)

- 指派 (Assignment): 给每个命题变元赋予真值 T 或 F
- n 个命题变元有 2^n 种可能的指派
- 对任意公式 A, 指派 φ 下 A 的真值记为 $\varphi(A)$

重要概念

- 永真式 (Tautology): 在所有指派下均为真
- 可满足式 (Satisfiable): 存在某指派使其为真
- 不可满足式 (Unsatisfiable): 所有指派下均为假
- 逻辑等价 (Equivalence): $A \equiv B$, 当且仅当 $A \leftrightarrow B$ 为永真式
- 逻辑蕴涵 (Entailment): $A \models B$, 当且仅当 $A \rightarrow B$ 为永真式

范式

- 合取范式 (CNF): 子句的合取, 如 $(P \vee Q \vee R) \wedge (\neg P \vee Q)$
- 析取范式 (DNF): 文字的合取的析取
- 任一命题公式都可等价转换为 CNF 或 DNF

2.4 命题演算 (Propositional Calculus, PC)

公理系统

- **AX1**: $A \rightarrow (B \rightarrow A)$
- **AX2**: $(A \rightarrow (B \rightarrow C)) \rightarrow ((A \rightarrow B) \rightarrow (A \rightarrow C))$
- **AX3**: $(\neg A \rightarrow \neg B) \rightarrow (B \rightarrow A)$
- 推理规则 (**MP**): 由 A 和 $A \rightarrow B$ 可推出 B

核心定理

- 可靠性 (Soundness): 若 $PC \vdash A$, 则 $\models A$ (可证的必为真)
- 完备性 (Completeness, Gödel 1929): 若 $\models A$, 则 $PC \vdash A$ (为真的必可证)
- 一致性 (Consistency): 不存在公式 A 使得 $PC \vdash A$ 且 $PC \vdash \neg A$
- 演绎定理 (Deduction Theorem): $\{A\} \vdash_{PC} B$ 当且仅当 $\vdash_{PC} A \rightarrow B$

2.5 一阶谓词逻辑 (First-Order Logic, FOL)

为什么需要 FOL?

- 命题逻辑表达能力有限 (不能表达"所有"、"存在"等量词关系)
- 如 "所有人都会死" 无法在命题逻辑中简洁表达

语法新增元素

- 个体词: 常量 ($a, b, c, \dots$)、变量 ($x, y, z, \dots$)
- 谓词 (Predicate): $P(x), Q(x, y)$, 表示个体的属性或关系
- 函数 (Function): $f(x), g(x, y)$, 映射个体到个体
- 全称量词 ($\forall$): $\forall x P(x)$ — "对所有 x "
- 存在量词 ($\exists$): $\exists x P(x)$ — "存在 x "

例子

- "凡是青蛙都是绿色的": $\forall x (Frog(x) \rightarrow Green(x))$
- "有人喜欢猫": $\exists x Likes(x, cat)$
- "所有人喜欢猫": $\forall x Likes(x, cat)$

量词辖域与变量

- 自由变量 vs 约束变量
- 量词对偶性: $\neg \forall x P(x) \equiv \exists x \neg P(x)$; $\neg \exists x P(x) \equiv \forall x \neg P(x)$

2.6 置换 (Substitution) ★

定义: $\sigma = \{t_1/x_1, t_2/x_2, \dots, t_n/x_n\}$, 其中 x_i 为变量, t_i 为项

应用: $E\sigma$ 表示将表达式 E 中的每个 x_i 同时替换为 t_i

示例:

- $P(x) \{apple/x\} = P(apple)$
- $Likes(x, icecream) \{John/x\} = Likes(John, icecream)$

2.7 FOL 推理规则

规则	形式
假言推理	$P, P \rightarrow Q \Rightarrow Q$
全称例示 (UI)	$\forall x P(x) \Rightarrow P(y)$ (y 为任意项)
存在例示 (EI)	$\exists x P(x) \Rightarrow P(y)$ (y 为新常量)

2.8 形式证明实例

例题: 给定以下前提, 证明 $\exists x (G(x) \wedge \neg D(x))$

- $\forall x (G(x) \rightarrow A(x) \vee B(x) \vee C(x))$
- $\forall x (G(x) \wedge D(x) \rightarrow \neg A(x))$
- $\forall x (G(x) \rightarrow \neg B(x) \wedge \neg C(x))$

(详细证明步骤见课件, 涉及全称例示、假言推理等规则)

复习要点

- 掌握命题逻辑的语法和语义
- 理解 tautology/satisfiable/unsatisfiable 的区别
- 能将自然语言转换为 FOL 公式
- 掌握置换的概念与应用
- 理解 FOL 的推理规则
- 与第三、四讲衔接: 本讲为归结原理做铺垫

第三、四讲：知识表示与推理 II —— 归结原理

3.1 命题逻辑（回顾与深化）

本讲承接第二讲, 重点在归结原理这一自动定理证明方法

逻辑联结词（复习）

- 否定 ($\neg$): NOT
- 合取 ($\wedge$): AND
- 析取 ($\vee$): OR
- 蕴含 ($\rightarrow$): IF...THEN, 等价于 $\neg P \vee Q$
- 双条件 ($\leftrightarrow$): 当且仅当

重要概念

- 合式公式 (WFF): Well-Formed Formula
- 永真式 (Tautology): 在所有解释下均为真
- 等价 (Equivalence): $P \equiv Q$
- 可满足性 (Satisfiability)

- 逻辑蕴涵 (Entailment): $KB \models \alpha$ — 在所有使 KB 为真的模型中, α 也为真

3.2 一阶谓词逻辑 (FOL) (深化)

语法元素

- 常量 (Constants): $a, b, c, \dots$
- 变量 (Variables): $x, y, z, \dots$
- 谓词 (Predicates): $P(x), Q(x,y), R(x,y,z), \dots$
- 函数 (Functions): $f(x), g(x,y), \dots$
- 全称量词 ($\forall$): "对所有 x "
- 存在量词 ($\exists$): "存在 x "
- 项 (Term): 常量、变量或函数作用于项

重要概念

- 自由变量 vs 约束变量
- 量词辖域 (Scope)
- 解释 (Interpretation) 与 模型 (Model)
- $KB \models \alpha$: 知识库 KB 蕴涵公式 α
- 可靠性 (Soundness): $KB \vdash \alpha \Rightarrow KB \models \alpha$
- 完备性 (Completeness): $KB \models \alpha \Rightarrow KB \vdash \alpha$

3.3 归结原理 (Resolution Principle) ★

J.A. Robinson, 1965 — 自动化定理证明的核心方法

子句形式 (Clause Form)

- 文字 (Literal): 原子公式或其否定
- 子句 (Clause): 文字的析取
- 合取范式 (CNF): 子句的合取
- 空子句 (NIL): 代表矛盾/假

命题归结规则

给定: $C_1 = L \vee C_1'$
 $C_2 = \neg L \vee C_2'$
归结: $C = C_1' \vee C_2'$ (归结式 Resolvent)

- L 和 $\neg L$ 为互补文字
- C_1 和 C_2 为父子句

反证法证明步骤

- 将 $\neg\alpha$ 加入 KB (否定目标)
- 将 $KB \cup \{\neg\alpha\}$ 化为子句集 S
- 反复应用归结规则导出新子句
- 若导出 **NIL**, 则原命题得证 (反证法)

归结策略

策略	特点
水平饱和 (Level Saturation)	逐层生成所有归结式，完备但低效
支撑集策略 (Set of Support)	至少一个父辈来自目标否定，完备且聚焦
线性归结 (Linear Resolution)	每次使用最新导出的子句，命题逻辑中完备
输入归结 (Input Resolution)	一个父辈来自原始子句集，不一定完备

答案提取 (Answer Extraction)

- 添加 $\neg \text{Goal}(x) \vee \text{Answer}(x)$ 到子句集
- 归结导出 NIL 时读取 $\text{Answer}(\dots)$ 中的绑定

3.4 合一算法 (Unification) ★

置换 (Substitution)

- $\sigma = \{t_1/v_1, t_2/v_2, \dots, t_n/v_n\}$
- 应用: $E\sigma$ = 将 E 中的每个 v_i 同时替换为 t_i
- 组合: $\sigma\theta$ = 先应用 σ 再应用 θ
- $\sigma\theta \neq \theta\sigma$ (一般不可交换)

MGU (最一般合一)

- σ 是 MGU 当且仅当: 对任何其他合一 θ , 存在 λ 使得 $\theta = \sigma\lambda$
- 合一算法步骤:
 - $k=0, F_k=F, \sigma_k=\epsilon$ (空置换)
 - 若 F_k 为单元素集 (所有表达式相同), 返回 σ_k
 - 找 F_k 的分歧集 D_k
 - 在 D_k 中找变量 x_k 和项 t_k (**occurs check**: x_k 不出现在 t_k)
 - $F_{k+1} = F_k\{t_k/x_k\}, \sigma_{k+1} = \sigma_k \circ \{t_k/x_k\}, k=k+1$, 转步骤2
 - 若找不到, 报告不可合一

3.5 子句形转换 (Skolem 化) ★

将任意 FOL 公式转化为 CNF 子句集的完整流程:

步骤	操作	说明
1	消去 $\rightarrow$	$P \rightarrow Q \rightarrow \neg P \vee Q$
2	缩小 $\neg$ 辖域	De Morgan 定律
3	变量标准化	每个量词使用不同变量名
4	Skolem 化	消去 $\exists$: 若在 $\forall x_1 \dots \forall x_n$ 辖域内, 用 Skolem 函数 $f(x_1, \dots, x_n)$ 替换; 否则用 Skolem 常量
5	移至前束范式	所有 $\forall$ 移至最前
6	分配 $\vee$ 对 $\wedge$	化 CNF
7	省略 $\forall$	化为子句集
8	再次变量标准化	各子句变量名独立

复习要点

- 能做命题逻辑和一阶逻辑的归结证明
- 熟练掌握合一算法 (含 occurs check)
- 能将 FOL 公式完整转化为子句集 (含 Skolem 化)
- 理解归结的可靠性与反证完备性
- 掌握归结算例 (如 Superman 谜题)

第五、六讲：搜索技术 I —— 盲目搜索与启发式搜索

3.1 搜索问题形式化

五要素

1. 状态空间 (State Space): 所有可能状态
2. 动作/后继函数 $S(x)$: 从状态 x 可执行的操作
3. 初始状态 (Initial State)
4. 目标条件 (Goal Test)
5. 启发式函数 $h(n)$ (用于启发式搜索)

解的定义

从初始状态到目标状态的动作序列

典型例题

- 罗马尼亚旅行: $\text{Arad} \rightarrow \text{Bucharest}$, 状态=城市, 动作=驾车
- 水壶问题: 3加仑和4加仑水壶, 状态= $(\text{gal3}, \text{gal4})$

- 八数码 (8-Puzzle): 3×3 滑块拼图
- 吸尘器世界: 8个房间, left/right/suck 动作

3.2 Frontier（Open List）与搜索框架

- **Frontier** = 待扩展节点的集合
- 初始: Frontier = {初始状态}
- 每步: 从 Frontier 取一节点 → 扩展 → 子节点加入 Frontier
- 关键: Frontier 的选择顺序决定了求解与否、解的质量、时空复杂度

路径检查 vs 循环检查

- 路径检查: 仅检查当前路径上是否出现重复
- 循环检查: 维护 Closed 表, 记录所有已扩展节点

3.3 盲目搜索策略

算法	Frontier 顺序	完备性	最优性	时间复杂度	空间复杂度
BFS 广度优先	FIFO 队列	☑	☑(等代价)	$O(b^{d+1})$	$O(b^{d+1})$
DFS 深度优先	LIFO 栈	✗(可循环)	✗	$O(b^m)$	$O(b^m)$
UCS 一致代价	按 $g(n)$ 优先	☑	☑	$O(b^{(C^*/s+1)})$	$O(b^{(C^*/s+1)})$
DLS 深度限制	DFS+L限深	✗	✗	$O(b^L)$	$O(b^L)$
IDS 迭代加深	重复DLS递增L	☑	☑(等代价)	$O(b^d)$	$O(b^d)$
Bidirectional	双向搜索	☑	☑	$O(b^{(d/2)})$	—

符号: b =分支因子, d =最浅目标深度, m =最大深度, L =深度限制, C^* =最优解代价, s =最小步代价

BFS 详细分析

- 逐层扩展, Frontier: $\{0\} \rightarrow \{1,2\} \rightarrow \{2,2,3\} \rightarrow \dots$
- 时间: $1+b+b^2+\dots+b^d+b(b^{d-1}) = O(b^{d+1})$
- 空间: $b(b^{d-1}) = O(b^{d+1})$
- 示例: $b=10, d=6 \rightarrow 18$ 分钟, 111MB; $d=8 \rightarrow 31$ 小时, 11GB

DFS 详细分析

- 时间: $O(b^m)$, 空间: **$O(b^m)$** (主要优势)
- 不完备 (若状态空间含环)
- DLS 伪代码:

若 $\text{Depth}(n) < D$: Frontier = (Frontier \ {n}) $\cup$ Successors(curr)
 否则: Frontier = Frontier \ {n}, CutOffOccurred = TRUE

IDS = DFS 空间 + BFS 完备性

- 时间: $(d+1)b^0 + d \cdot b^1 + (d-1) \cdot b^2 + \dots + b^d = O(b^d)$
- 仅比 BFS 略慢, 但空间为 $O(bd)$

3.4 启发式搜索

Greedy Best-First Search

- $f(n) = h(n)$ (仅启发式)
- 扩展"看起来最近的"节点
- ✗ 不完备, 不保证最优

A* 搜索 ★

- $f(n) = g(n) + h(n)$
 - $g(n)$ = 从起点到 n 的实际代价
 - $h(n)$ = 从 n 到目标的启发式估计
 - $f(n)$ = 经过 n 的最优解的估计代价
- 统一了 UCS (优化 g) 和 Greedy (优化 h)

A* 最优性条件

1. 可采纳性 (Admissibility)

- $h(n) \leq h^*(n)$, 对所有 n (永不高估真实代价)
- $h(n)=0$ 总是可采纳的 (退化为 UCS)
- 可采纳性 $\Rightarrow$ A* 最优

2. 一致性/单调性 (Consistency/Monotonicity)

- $h(n_1) \leq c(n_1, n_2) + h(n_2)$ (三角不等式)
- 一致性 $\Rightarrow$ 可采纳性 (更强条件)
- 一致性 $\Rightarrow f$ 值沿路径非递减

A* vs IDA*

- A* 需要存储所有节点 (空间大)
- IDA* = 迭代加深 + A* (用 $f=g+h$ 作为截断值)
- 每次迭代提高截断值至上次超过的最小 f 值

3.5 启发式设计: 八数码

启发式	定义	性质
h_1	不在位棋子数	可采纳
h_2	曼哈顿距离 (各棋子到目标位置水平和垂直距离之和)	可采纳, h_2 支配 h_1

性能对比 (展开节点数)

深度	IDS	$A^*(h_i)$	$A^*(h_2)$
10	47,127	93	39
14	3,473,941	539	113
24	—	39,135	1,641

结论: 更好的启发式（支配性越强）极大缩减搜索空间。

📖 复习要点

- 能对给定问题形式化搜索（五要素）
- 掌握 BFS/DFS/UCS/DLS/IDS 的性质、复杂度
- 理解 A^* : $f=g+h$, 可采纳性与一致性的定义与作用
- 能判断启发式是否可采纳/一致
- 理解 IDS/IDA* 的思想

第七讲：搜索技术 II —— 对抗搜索与博弈

4.1 博弈 AI 历史

年份	事件
1912	Zermelo — MiniMax 理论基础
1949	Shannon — 国际象棋编程
1956	McCarthy — AI 与棋类
1958	Alpha-Beta 剪枝; IBM 704 象棋程序
1997	IBM Deep Blue 击败 Kasparov（每秒评估2亿局面）
2007	西洋跳棋被解决 (Schaeffer, <i>Science</i>)
2016	AlphaGo 击败李世石 (<i>Nature</i>)
2017	AlphaGo Zero（零人类知识自对弈）; AlphaZero（通用）
2019	OpenAI Five (Dota 2)

4.2 博弈基本概念

- 零和博弈: 一方所得即另一方所失
- **MAX** 节点: 我方行动, 选择最大值
- **MIN** 节点: 对方行动, 选择最小值
- 终局状态: 已知效用值

- 博弈树: MAX 和 MIN 交替的树结构

4.3 MiniMax 算法

```
DFMiniMax(n, Player):  
    If n is TERMINAL: Return U(n)  
    ChildList = Successors(Player)  
    If Player is MIN: return minimum of DFMiniMax(c, MAX)  
    Else: return maximum of DFMiniMax(c, MIN)
```

性质:

- ☒ 对完美对手最优
- 时间: $O(b^d)$, 空间: $O(bd)$
- 国际象棋: $b \approx 35$, $d \approx 50 \Rightarrow$ 完全不可行

4.4 Alpha-Beta 剪枝 ★

核心思想: 剪除不影响最终决策的分支

- α (Alpha): MAX 目前已见的最佳选择值 (下界)。初始 = $-\infty$
- β (Beta): MIN 目前已见的最佳选择值 (MAX 的上界)。初始 = $+\infty$
- 剪枝条件: 若 $\alpha \geq \beta$, 剪枝 (停止搜索该分支)

效率:

- 最优排序下: $O(b^{(d/2)})$ — 相当于搜索深度翻倍
- Deep Blue: 实际分支因子从 35 降至 ~ 6

4.5 评估函数与资源限制

- 不能搜索到终局时: 使用评估函数评估非终局局面
- 深度是关键: 评估函数藏在越深处, 其质量影响越小
- 迭代加深: 持续加深搜索直到时间耗尽
- 国际象棋评估: 兵=1, 马/象=3, 车=5, 后=9, Deep Blue 用 ~ 8000 个特征

4.6 蒙特卡洛树搜索 (MCTS)

四阶段循环:

1. 选择 (Selection): 用 UCB1 公式从根节点选择到未扩展节点
2. 扩展 (Expansion): 加入一个新子节点
3. 模拟 (Simulation/Rollout): 随机下到终局
4. 回传 (Backpropagation): 更新访问次数和胜率

4.7 AlphaGo 架构

训练三阶段:

1. 行为克隆 (Supervised Learning): 用人类棋谱初始化策略网络 (交叉熵损失)
2. 策略梯度 (RL): 策略网络自我对弈 (REINFORCE, 赢+1/输-1)
3. 价值网络 (Regression): 预测状态价值 $V(s)$

执行: MCTS 结合策略网络和价值网络

复习要点

- 理解 MiniMax 原理与伪代码
- 熟练掌握 Alpha-Beta 剪枝 (α/β 含义、剪枝条件、走一遍实例)
- 理解 MCTS 四阶段
- 评估函数的作用与深度权衡

第八讲：高级搜索技术 —— 局部搜索与优化

5.1 局部搜索概述

- 不关心到达解的路径，只关心解的状态本身
- 适用于优化问题：在状态空间中寻找最优状态
- 不维护 Frontier/Closed 表，内存开销极低

5.2 爬山算法 (Hill-Climbing)

```
function Hill-Climbing(problem):
    current = Make-Node(Initial-State[problem])
    loop:
        neighbor = highest-valued successor of current
        if Value[neighbor] < Value[current]:
            return State[current]    // 到达局部最优
        current = neighbor
```

特点:

- 贪心策略，选择最佳后继
- ✗ 不保证全局最优（陷入局部极值、高原、山脊）
- 变体: 随机爬山、首选爬山、随机重启爬山

5.3 模拟退火 (Simulated Annealing)

- Kirkpatrick et al., 1983 (Metropolis, 1953)
- 模拟金属退火过程，允许以一定概率接受更差的解
- 接受概率: **Boltzmann** 分布 $e^{(-\Delta E/T)}$
- 温度 **T** 从高到低逐渐下降（冷却计划）
- 高温时探索空间广，低温时收敛到最优

参数:

- 初始温度、冷却率、终止温度
- 适用于 **NP 难** 问题的近似求解

5.4 八数码问题与启发式

- h_1 = 不在位棋子数

- h_2 = 曼哈顿距离 (Manhattan distance)
- 曼哈顿距离对八数码始终 $\geq$ 不在位数

复习要点

- 爬山算法的伪代码与局限性
- 模拟退火的核心思想（接受概率、温度的作用）

第九讲：概率推理与不确定性

6.1 不确定性概述

不确定性的来源

- 部分可观测性
- 动作的非确定性效果
- 传感器噪声
- 模型不完备

处理方法

- 概率论: 处理不确定性的标准数学框架
- 模糊逻辑 (Fuzzy Logic): Lotfi Zadeh, 1965
- 确定性因子 (Certainty Factors): Shortliffe, 1976 (MYCIN 系统)

6.2 概率论基础

- 先验概率 $P(A)$
- 条件概率 $P(A|B) = P(A \cap B) / P(B)$
- 联合分布 (Joint Distribution)
- 贝叶斯规则 (Bayes' Rule): $P(A|B) = P(B|A) \cdot P(A) / P(B)$

6.3 决策理论

期望效用最大化 (Maximum Expected Utility, MEU)

- 行动选择: $a^* = \operatorname{argmax}_a \sum P(\text{结果} | a) \cdot U(\text{结果})$
- 效用函数 U : 量化对不同结果的偏好

6.4 贝叶斯网络 (Bayesian Networks)

- Judea Pearl (2011 图灵奖)
- 有向无环图 (DAG) 表示变量间的条件依赖关系
- 节点 = 随机变量, 边 = 概率依赖
- 信念传播 (Belief Propagation, BP): Pearl, 1988

相关模型

- 隐马尔可夫模型 (HMM)
- 马尔可夫随机场 (MRF)

- 条件随机场 (CRF)

 复习要点

- 掌握贝叶斯规则及其应用
- 理解期望效用最大化 (MEU) 概念
- 了解贝叶斯网络的基本结构

第十讲：决策树

7.1 决策树算法家族

算法	发明者/年份	特征
CLS	Hunt, Marin, Stone, 1966	概念学习系统，最早
ID3	Quinlan, 1979/1986	信息增益
C4.5	Quinlan, 1993	增益率 (Gain Ratio)
CART	Breiman et al., 1984	Gini 指数

7.2 决策树基本概念

- 节点: 对应一个属性测试
- 分支: 对应属性的可能取值
- 叶节点: 对应分类结果
- 构建: 递归选择最优划分属性
- 剪枝: 防止过拟合

7.3 信息熵 (Entropy)

$$H(X) = - \sum_{i=1}^n P(x_i) \log_2 P(x_i)$$

(1)

- 衡量随机变量的不确定性
- 对于二分类: $H(p) = -p \cdot \log_2(p) - (1-p) \cdot \log_2(1-p)$
- $p=0.5$ 时熵最大 (1 bit)

7.4 条件熵与互信息

条件熵:

$$H(Y|X) = \sum_x P(x) \cdot H(Y|X = x)$$

(2)

互信息 (信息增益):

$$I(X;Y) = H(Y) - H(Y|X) = H(X) - H(X|Y) \quad (3)$$

- ID3 使用信息增益选择划分属性
- 问题: 偏向取值多的属性

7.5 C4.5 — 增益率 (Gain Ratio)

$$\text{GainRatio}(A) = \frac{\text{Gain}(A)}{\text{SplitInfo}(A)} \quad (4)$$

$$\text{SplitInfo}(A) = - \sum_{v=1}^V \frac{|D_v|}{|D|} \log_2 \left(\frac{|D_v|}{|D|} \right) \quad (5)$$

- 解决了信息增益偏向多值属性的问题

7.6 CART — Gini 指数

$$\text{Gini}(D) = 1 - \sum_{k=1}^K p_k^2 = \sum_{k=1}^K p_k (1 - p_k) \quad (6)$$

- 衡量数据集的纯度/不纯度
- 选择使 Gini 指数最小的属性
- 支持二叉分裂

7.7 连续值处理

- 排序后取相邻值中点作为候选分裂点
- 常用策略: 二分法 (取中值), 即选取第 $(v_i + v_{i+1})/2$ 个候选点

复习要点

- 能手工计算信息熵、信息增益、增益率、Gini 指数
- 理解 ID3/C4.5/CART 的区别与联系
- CLS 算法基本流程
- 理解决策树构建与剪枝

第十一讲：机器学习基础

8.1 机器学习定义

- **Arthur Samuel (1959):** "使计算机无需显式编程就能学习的研究领域"
- **Tom Mitchell:** 从经验 E 中学习任务 T , 性能度量 P 随经验改进
- **Herbert A. Simon:** 1975 图灵奖 / 1978 诺贝尔经济学奖

8.2 四大学习范式

范式	数据	任务
监督学习 (Supervised)	数据 + 标签 $\{(x_i, y_i)\}$	学习 $f: x \rightarrow y$ (分类/回归)
半监督学习 (Semi-Supervised)	少量标签 + 大量无标签 $\{x_1, \dots, x_m, (x_{m+1}, y_{m+1}), \dots\}$	利用无标签数据改进
无监督学习 (Unsupervised)	仅数据 $\{x_i\}$	发现隐藏结构 (聚类/降维)
强化学习 (Reinforcement)	状态-动作-奖励 (s, a, r)	学习最优策略 π

8.3 ML 三大组件

- 1. 表示 (Representation): 数据如何指定? 模型什么形式? (CNN, RNN, Transformer, 决策树...)
- 2. 优化 (Optimization): 模型如何训练? (梯度下降, Adam, 遗传算法...)
- 3. 评估 (Evaluation): 如何衡量成功?
 - o 准确率 (Accuracy)、均方误差 (Squared Error)
 - o 精确率/召回率 (Precision/Recall)、F-measure
 - o 似然 (Likelihood)、后验概率 (Posterior)
 - o KL 散度 (KL Divergence)

8.4 ML 设计流程

训练数据 → 学习器 → 知识/模型
测试数据 → 性能评估

- **i.i.d.** 假设: 训练与测试数据独立同分布
- 若非独立 → 集体分类 (Collective Classification)
- 若分布不同 → 迁移学习 (Transfer Learning)

8.5 案例实践

- 从用户画像 → 特征向量
- 获取标注数据 → 形成数据矩阵 X 和标签向量 y
- 训练模型 → 应用模型
- 示例: ChatGPT, MidJourney, 自动驾驶, InstructTTS, 医学影像

复习要点

- 四种学习范式的区别与特征
- 分类/回归/生成任务的区别
- ML 三组件 (表示、优化、评估)
- i.i.d. 假设及其重要性

第十二讲：无监督学习 —— 聚类

9.1 聚类基本概念

目标: 组内距离最小化，组间距离最大化

应用: 市场细分、文档聚类、降水模式发现、行业分组

9.2 聚类类型分类

分类维度	类型
是否划分	划分聚类 (Partitional) vs 层次聚类 (Hierarchical)
隶属关系	硬聚类 vs 模糊/概率聚类 (每个点以权重属于各类)

簇的类型:

- 分离良好的簇 (Well-Separated)
- 中心型簇 (Center-based)
- 连续型簇 (Contiguous)
- 密度型簇 (Density-based)
- 概念/属性型簇 (Conceptual)

9.3 K-means 算法 ★

算法步骤:

1. 随机选择 K 个初始中心
2. 将每个点分配给最近的中心
3. 重新计算每个簇的中心（均值）
4. 重复 2-3 直到收敛

复杂度: $O(n \cdot K \cdot l \cdot d)$

目标函数 — SSE (误差平方和):

$$SSE = \sum_{i=1}^K \sum_{x \in C_i} \text{dist}^2(m_i, x)$$

(7)

K 值选择:

- 肘部法则 (Elbow Method): 画 SSE-K 曲线，找"肘点"

改进:

- K-means++: 用 $D(x)^2$ 加权采样初始中心（保证 $O(\log k)$ 近似比）
- K-means||: 并行初始化

局限: 不同大小/密度/非球形/含离群点的聚类效果差

9.4 DBSCAN 算法 ★

核心定义:

- 核心点 (Core Point): Eps 邻域内 $\geq \text{MinPts}$ 个点
- 边界点 (Border Point): 在核心点邻域内但不满足 MinPts
- 噪声点 (Noise): 既非核心也非边界

算法:

1. 对每个未访问的核心点，创建一个新簇
2. 将该核心点的所有密度可达点加入簇
3. 标记非核心非边界点为噪声

优点: 抗噪声、处理任意形状簇、不需指定 K 局限: 变化密度和高维数据效果差

参数选择: k-dist 图（第 k 近邻距离排序图）找拐点

9.5 K-means vs DBSCAN

特性	K-means	DBSCAN
噪声处理	使用所有数据	丢弃噪声
类型	基于原型	基于密度
簇形状	各向同性	任意形状
K 值	需预设	自动确定

9.6 聚类评估

- 外部指标: 与真实标签比较（熵）
- 内部指标: SSE、轮廓系数
- $TSS = SSE + SSB$ （总平方和 = 组内 + 组间，为常数）

复习要点

- K-means 算法步骤、SSE、肘部法则
- K-means++ 初始化原理
- DBSCAN 的核心点/边界点/噪声定义
- 两种算法的对比与适用场景

第十三讲：人工神经网络

10.1 生物启发

- 大脑 ~1000 亿神经元
- 神经元结构: 树突 $\rightarrow$ 细胞体 $\rightarrow$ 轴突

10.2 MP 神经元 (McCulloch-Pitts, 1943)

$$z = \sum_i w_i \cdot a_i = \mathbf{w}^T \mathbf{a} \quad (8)$$

$$\text{output} = \text{sgn}(z) = \begin{cases} 1, & z \geq 0 \\ -1, & z < 0 \end{cases} \quad (9)$$

- 最早的人工神经元模型
- **Hebb** 学习规则 (1949): "一起放电的神经元连在一起" (Cells that fire together, wire together)

10.3 感知机 (Perceptron)

- MP 神经元的可学习版本
- 单层输出, 学习规则: $\mathbf{w} \leftarrow \mathbf{w} + (y - \hat{y}) \cdot \mathbf{x}$
- **Minsky 1969**: 感知机不能解决 **XOR** 问题 (线性不可分)
- 权重定义超平面: $\mathbf{w}^T \mathbf{a} = 0$

10.4 多层感知机 (MLP) ★

架构: 输入层 → 隐藏层(≥1) → 输出层

$$\mathbf{y} = \sigma_2(\mathbf{W}_2 \cdot \sigma_1(\mathbf{W}_1 \cdot \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) \quad (10)$$

- 可解决 XOR 问题

10.5 激活函数

函数	公式	导数	使用场景
sgn	$\text{sgn}(x) = \{1, x \geq 0; -1, x < 0\}$	0 (除原点)	MP 神经元
Sigmoid	$\sigma(x) = 1/(1+e^{-x})$	$\sigma(x)(1-\sigma(x))$	早期 MLP
tanh	$\tanh(x) = (e^x - e^{-x})/(e^x + e^{-x})$	$1 - \tanh^2(x)$	DNN
ReLU	$\max(0, x)$	$\{0, x < 0; 1, x > 0\}$	现代 DNN

10.6 万能逼近定理

含单隐层的前馈网络可以逼近任何连续函数 (Cybenko 1989, Hornik et al. 1989)

10.7 前向传播计算示例

给定输入 $a_1=0.05, a_2=0.10$, 偏置 $b=0.35$, 权重 w :

- $z_1 = w_1 \cdot a_1 + w_2 \cdot a_2 + b$ (隐层求和)
- $o_1 = \sigma(z_1)$ (Sigmoid 激活)
- Loss: $L = \frac{1}{2} \sum (\text{target} - \text{output})^2$

10.8 反向传播 (Backpropagation) ★

Rumelhart, Hinton, 1986

链式法则:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial z} \cdot \frac{\partial z}{\partial w} \tag{11}$$

Sigmoid 导数: $\partial \sigma / \partial z = \sigma(z) \cdot (1 - \sigma(z))$

权重更新: $w^{(new)} = w^{(old)} - \eta \cdot \partial L / \partial w$

 复习要点

- MP 神经元模型
- XOR 问题的线性不可分性
- MLP 架构与前向传播公式
- 四种激活函数及其导数
- 反向传播的链式法则推导（重要计算考点）
- 理解万能逼近定理的含义

第十四讲：深度学习 I —— CNN/RNN/LSTM

11.1 从 MLP 到深度网络

$$y = \sigma_L(\mathbf{W}_L \cdots \sigma_2(\mathbf{W}_2 \cdot \sigma_1(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) \cdots + \mathbf{b}_L) \tag{12}$$

11.2 ImageNet 竞赛历程

年份	模型	层数	Top-5 错误率
2012	AlexNet	8	16.4%
2014	VGG	19	7.3%
2014	GoogleNet (Inception)	22	6.7%
2015	ResNet	152	3.57%

2018 图灵奖: Bengio, Hinton, LeCun（深度学习先驱）

11.3 CNN（卷积神经网络）★

为什么 **CNN**?

- MLP 将所有像素独立处理 → 忽略空间结构
- CNN 参数共享（权重在空间位置间共享）→ 大幅减少参数

核心操作

1. 卷积 (Convolution):

- 滤波器/卷积核 (Filter/Kernel) 在图像上滑动
- 输出特征图 (Feature Map)
- 步长 (Stride) = 1 或 2

2. Padding:

- **Same:** 输出尺寸 = 输入尺寸 / 步长
- **Valid:** 无填充
- 输出尺寸 = (n - f + 2p)/s + 1

3. 最大池化 (Max Pooling):

- 降采样操作
- 在每块区域内取最大值
- 典型模式: Conv → MaxPool → Conv → MaxPool → ... → FC → Output

CNN vs MLP 对比

- CNN: 局部连接 + 参数共享 → 参数少
- MLP: 全连接 → 参数多

11.4 RNN（循环神经网络）

目的: 处理序列数据（文本、语音、时间序列）

$$h_t = f(\mathbf{W} \cdot h_{t-1} + \mathbf{U} \cdot x_t + b) \tag{13}$$

$$y_t = \mathbf{V} \cdot h_t \tag{14}$$

BPTT (Back-Propagation Through Time): 梯度沿时间反向传播

梯度消失/爆炸问题

- w=0.99, $w^{1000} \rightarrow 0$ （消失）
- w=1.01, $w^{1000} \rightarrow 20000$ （爆炸）
- 对策: 梯度裁剪 (Gradient Clipping)

11.5 LSTM（长短期记忆网络）★

四个门控 + 记忆单元:

组件	作用
遗忘门 (Forget Gate)	决定丢弃哪些旧信息
输入门 (Input Gate)	决定存储哪些新信息
输出门 (Output Gate)	决定输出什么
记忆单元 c_t	长期记忆通道

- 门控用 **Sigmoid** (0~1), 候选值用 **tanh** (-1~1)
- 有效缓解梯度消失问题

11.6 其他主题

- 对抗样本 (Adversarial Examples): 深度网络容易被微小扰动欺骗
- 可解释 AI (Explainable AI): 局部解释 (为什么这张图是猫?) vs 全局解释 (猫在模型眼中什么样?)
- 生成模型: GAN, VAE 等
- 关键架构: Transformer, BERT, ChatGPT, AlphaFold

复习要点

- CNN 的核心操作 (卷积、池化、Padding)
- 参数共享与局部连接的优势
- RNN 的 BPTT 与梯度消失/爆炸问题
- **LSTM** 的四个门控机制 (重点理解)
- 深度学习发展里程碑

第十五讲：深度学习 II —— Transformer

15.1 自然语言处理 (NLP) 基础

为什么 **NLP** 需要特殊处理?

- 语言的离散性和组合性
- 同义词、多义词、语境依赖
- 长距离依赖关系

早期方法

- **WordNet**: 人工构建的词汇数据库 (同义词集、上位词 "is-a" 关系)
 - **✗** 问题: 覆盖不全、无法捕捉语境、维护成本高

15.2 词嵌入 (Word Embedding)

One-Hot 编码的问题

- motel = [0,0,...,0,1,0,...,0], 维度 = 词汇量 (50万+)
- **✗** 无法度量相似性: "motel" 和 "hotel" 的 one-hot 向量正交

分布式语义假说 (**Distributional Hypothesis**)

"You shall know a word by the company it keeps." — J.R. Firth, 1957

- 核心思想: 词的含义由其上下文决定
- 通过统计词共现关系学习稠密向量表示 (50-300维)

15.3 SVD 词向量

- 构建共现矩阵: 统计词对的窗口内共现次数

- 对共现矩阵做 SVD 分解降维
- 通过 LLE 等算法可视化

15.4 Word2Vec ★

Mikolov et al., 2013 — "Distributed Representations of Words and Phrases"

核心思想

- 训练模型根据上下文预测目标词（或反过来）
- 学习到高质量的低维词向量

CBOW (Continuous Bag of Words)

- 任务: 用上下文词窗口预测中心词
- 输入: 上下文词的 one-hot 向量
- 过程: 上下文向量平均 $\rightarrow$ 隐藏层 $\rightarrow$ Softmax 输出目标词概率
- 架构: $\text{Input}(H \times V) \times W(|V| \times N) \rightarrow \text{Hidden}(N) \times W'(N \times |V|) \rightarrow \text{Output}(|V| \times 1)$
- 损失: 交叉熵 (Cross Entropy)

Skip-Gram

- 任务: 用中心词预测上下文词窗口
- 与 CBOW 相反: "输入 $\rightarrow$ 周围词"
- 对于罕见词效果更好

关键发现

- **king - man + woman $\approx$ queen**（词向量可做类比推理）
- 词向量捕捉语法和语义规律

局限性

- 每个词只有一个向量，无法处理一词多义
- 如 "read" $\rightarrow$ "reading", "reader", "readable" 在 Word2Vec 中无关联

15.5 GloVe 与 FastText

GloVe (Global Vectors for Word Representation)

- 结合全局统计信息（共现计数）和局部上下文窗口
- 在共现矩阵上做矩阵分解

FastText (Facebook, 2016)

- 基于子词 (**subword**) 的 CBOW
- 将词拆分为 n-gram 字符序列
- 可处理 **OOV** (Out-of-Vocabulary) 词

15.6 机器翻译 (Machine Translation)

方法演变

年代	方法	特点
1990s-2010s	SMT (统计机器翻译)	短语对齐、语言模型，需要大量特征工程
2014-2016	NMT (神经机器翻译)	Seq2Seq+RNN，端到端训练
2016+	NMT 超越 SMT	NYT 报道 "The Great AI Awakening"

15.7 Seq2Seq (Encoder-Decoder) ★

架构

Encoder (编码器): 输入 $x \rightarrow$ 中间表示 (context vector)
 Decoder (解码器): 中间表示 $\rightarrow$ 输出 y

核心思想

- Encoder 将可变长输入压缩为固定长度的上下文向量
- Decoder 从上下文向量生成可变长输出

应用

- 机器翻译 (MT)
- 文本摘要 (Summarization)
- 对话生成 (Dialogue)
- NL2SQL
- 代码生成

15.8 注意力机制 (Attention) ★

为什么需要 Attention?

- ✗ Encoder-Decoder 的瓶颈: 所有信息压缩到单个固定长度向量
- ☑ Attention: 允许 Decoder 在每一步动态关注输入的不同部分

Attention 三步计算

- 相似度计算: $e_i = \text{similarity}(\text{query}, \text{key}_i)$, 常用点积 $\text{score}(q, k) = q \cdot k$
- 归一化: $\alpha_i = \text{softmax}(e_1, \dots, e_n)$ (注意力权重, 和为1)
- 加权求和: $\text{context} = \sum \alpha_i \cdot v_i$

QKV 框架

- Query** (查询): 当前想知道什么 (Decoder 状态)
- Key** (键): 输入的索引 (Encoder 各位置)
- Value** (值): 输入的内容 (Encoder 各位置)
- 输出 = $\sum (\text{Query 与 Key 的相似度}) \times \text{Value}$

Seq2Seq + Attention

- Decoder 每个时刻计算当前状态与所有 Encoder 状态的注意力
- 动态加权上下文向量参与解码

15.9 Transformer ★★

Vaswani et al., 2017 — "Attention Is All You Need" 彻底取代 RNN/CNN 成为 NLP 和更多领域的核心架构

Transformer 相比 RNN 的三大优势

优势	RNN	Transformer
计算复杂度	$O(n)$ 序列依赖（不可并行）	$n \ll d$ 时每层复杂度低
长距离依赖	信息需经过 $O(n)$ 步传递	自注意力 $O(1)$ 直连任意位置
并行化	序列依赖无法并行	完全可并行（GPU/TPU 友好）

自注意力 (Self-Attention) 核心公式

步骤 1 — 生成 QKV:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V \tag{15}$$

其中 $X \in \mathbb{R}^{(n \times d)}$, $W^Q, W^K, W^V \in \mathbb{R}^{(d \times d_k)}$

步骤 2 — 计算注意力分数:

$$\text{Score} = QK^T \tag{16}$$

步骤 3 — Scaled Dot-Product 归一化:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V \tag{17}$$

- 除以 $\sqrt{d_k}$ 避免点积值过大导致 softmax 梯度消失

步骤 4 — 加权输出:

$$\text{Output} = \text{Attention}(Q, K, V) \tag{18}$$

三大关键技巧 (Three Tricks)

Trick #1: 残差连接 (Residual Connections)

$$X^{(l)} = \text{Layer}(X^{(l-1)}) + X^{(l-1)} \tag{19}$$

- 缓解深层网络梯度消失问题
- 平滑 Loss Landscape

Trick #2: 层归一化 (Layer Normalization)

- 对每个样本的特征维度归一化（均值=0, 方差=1）
- 加速训练，稳定梯度

Trick #3: Scaled Dot-Product Attention

- 如前所述，除以 $\sqrt{d_k}$ 防止 Softmax 区域饱和

多头注意力 (Multi-Head Attention)

- 并行运行 h 个自注意力头（ h 个不同的 W^Q, W^K, W^V ）
- 各头关注不同的语义关系
- 拼接各头输出: $\text{MultiHead} = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^O$
- 每个头的维度 $d_k = d/h$

位置编码 (Positional Encoding)

- Transformer 无循环结构 → 无法感知位置信息
- 解法: 在输入嵌入中加上正弦位置编码 (Sinusoidal Positional Encoding)
- 不同频率的正弦/余弦函数 → 唯一的位置表示
- 或使用可学习的位置嵌入

Transformer 整体架构

编码器 (Encoder): $N \times$ 层

每层: Multi-Head Self-Attention → Add&Norm → FFN → Add&Norm

解码器 (Decoder): $N \times$ 层

每层: Masked Multi-Head Self-Attention → Add&Norm

→ Cross-Attention (Q from Decoder, K,V from Encoder) → Add&Norm

→ FFN → Add&Norm

- **Masked Self-Attention:** 防止 Decoder 看到未来信息（训练时）
- **Cross-Attention:** Decoder 的 Q 去查询 Encoder 的 K,V

15.10 Transformer 的影响

Scaling Law

- 模型参数量、数据量、计算量 → 性能可预测增长
- 催生 GPT、BERT、LLaMA、DeepSeek 等大模型

超越 NLP

- **ViT** (Vision Transformer): 图像识别
- **AlphaFold:** 蛋白质结构预测
- 多模态: CLIP, DALL·E, GPT-4V

复习要点

- 理解分布式语义假说和 Word2Vec (CBOW/Skip-Gram)
- 掌握 Attention 机制的 QKV 框架和计算
- ★ **Transformer Self-Attention** 公式（核心考点）

- 理解 Multi-Head Attention 和 Positional Encoding
- Seq2Seq + Attention → Transformer 的演进逻辑
- 三大 Tricks 的作用

第十六讲：强化学习

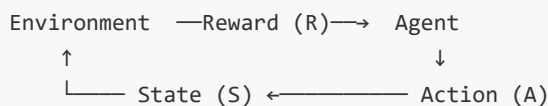
12.1 强化学习定义

Sutton 定义: "RL 是关于学习做什么——如何将状态映射到动作——以最大化数值奖励信号的问题"

RL 三大特点:

1. 本质上是闭环的（行动影响后续输入）
2. 没有直接指示应该采取什么行动
3. 行动的后果（包括奖励信号）在延长时间段内展开

12.2 RL 核心组件



- 状态 S_t : 当前环境状态
- 动作 A_t : Agent 的行为
- 奖励 R_t : 即时反馈信号
- 策略 $\pi(a|s)$: 状态到动作的映射（可为随机策略）
- 折扣因子 $\gamma \in [0,1]$: 未来奖励的折扣

奖励假设: 所有目标都可以通过最大化期望累积奖励来描述

12.3 马尔可夫决策过程 (MDP) ★

MDP 五元组: $\langle S, A, P, R, \gamma \rangle$

- 转移概率: $P(s'|s,a) = \Pr\{S_{t+1} = s' \mid S_t = s, A_t = a\}$
- 马尔可夫性质: 未来仅依赖于当前状态和动作，与历史无关

12.4 值函数

回报 (Return):

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (20)$$

状态价值函数 $V(s)$:

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s] \quad (21)$$

动作价值函数 $Q(s,a)$:

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a] \quad (22)$$

V 与 **Q** 的关系:

$$V_{\pi}(s) = \sum_a \pi(a|s) \cdot Q_{\pi}(s, a) \quad (23)$$

$$Q_{\pi}(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \cdot V_{\pi}(s') \quad (24)$$

12.5 Bellman 方程 ★

Bellman 期望方程:

$$V_{\pi}(s) = \sum_a \pi(a|s) \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) \cdot V_{\pi}(s') \right] \quad (25)$$

Bellman 最优方程:

$$V^*(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) \cdot V^*(s') \right] \quad (26)$$

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \cdot \max_{a'} Q^*(s', a') \quad (27)$$

12.6 动态规划方法

方法	说明
策略评估 (Policy Evaluation)	给定 π , 迭代计算 $V(s)$
策略改进 (Policy Improvement)	$\pi'(s) = \operatorname{argmax}_a Q_{\pi}(s, a)$
策略迭代 (Policy Iteration)	交替 评估 $\rightarrow$ 改进 至收敛
值迭代 (Value Iteration)	$V_{k+1}(s) = \max_a [R + \gamma \sum P \cdot V_k(s')]$

12.7 无模型方法

蒙特卡洛 (MC) 方法

- 从完整 Episode 中学习
- $V(s)$ = 从状态 s 开始的回报平均值
- 高方差、无偏

时序差分 (TD) 学习

TD(0):

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)] \quad (28)$$

- TD 目标: $R_{t+1} + \gamma V(S_{t+1})$
- TD 误差: $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$
- 结合 MC 和 DP 的优点

12.8 SARSA (On-Policy) vs Q-Learning (Off-Policy) ★

特性	SARSA	Q-Learning
类型	On-Policy	Off-Policy
更新	$Q(s,a) \leftarrow Q + \alpha[r + \gamma Q(s',a') - Q]$	$Q(s,a) \leftarrow Q + \alpha[r + \gamma \max_a Q(s',a) - Q]$
行为策略	ϵ -greedy (探索中学习)	ϵ -greedy (探索)
目标策略	同行为策略	贪心策略 (最优)

12.9 深度强化学习 (DRL)

DQN (Deep Q-Network)

- 用 DNN 近似 Q 函数
- 经验回放 (Experience Replay): 存储 (s,a,r,s') , 随机采样打破相关性
- 目标网络 (Target Network): 单独的网络计算目标值, 定期更新

DQN 变体:

- **Double DQN**: 解耦动作选择与评估, 减少过估计
- **Dueling DQN**: $Q(s,a) = V(s) + A(s,a)$, 分离状态价值与优势
- **Rainbow DQN**: 组合六种改进

策略梯度 (Policy Gradient)

- 直接参数化并优化策略 $\pi_\theta(a|s)$
- **REINFORCE**: $\theta \leftarrow \theta + \alpha \cdot \nabla_\theta \log \pi(a|s) \cdot G_t$
- 减基线 (Baseline) 降低方差

Actor-Critic 方法

- **Actor** (策略网络): 决定动作 $\pi_\theta(a|s)$
- **Critic** (价值网络): 评估动作 $V_w(s)$ 或 $Q_w(s,a)$
- **A2C** (Advantage Actor-Critic): 用优势函数 $A(s,a) = Q(s,a) - V(s)$

12.10 AlphaGo / AlphaGo Zero

训练三阶段 (已在第七讲详述):

1. 监督学习初始化
2. 强化学习自我对弈
3. 价值网络回归

MCTS 执行: 选择 (UCB) $\rightarrow$ 扩展 $\rightarrow$ 评估 (快速走子+价值网络) $\rightarrow$ 回传

12.11 强化学习方法总结

类别	代表算法
基于价值 (Value-based)	DQN, Double DQN, Dueling DQN, Rainbow
基于策略 (Policy-based)	REINFORCE
Actor-Critic	A2C, A3C
基于模型 vs 无模型	—
On-Policy vs Off-Policy	SARSA vs Q-Learning

复习要点

- MDP 五元组定义
- $V(s)$, $Q(s,a)$ 定义与关系
- **Bellman** 期望方程与最优方程（核心公式）
- TD 更新公式与 TD 误差
- **SARSA vs Q-Learning** 的区别（On-policy vs Off-policy）
- DQN 的经验回放与目标网络
- Actor-Critic 基本架构
- AlphaGo 的训练流程

附录：课程核心公式速查

搜索

- A^* : $f(n) = g(n) + h(n)$
- 可采纳性: $h(n) \leq h^*(n)$
- 一致性: $h(n_1) \leq c(n_1, n_2) + h(n_2)$

逻辑

- 归结: $(L \vee C_1') \wedge (\neg L \vee C_2') \Rightarrow C_1' \vee C_2'$

信息论

- 熵: $H(X) = -\sum p(x) \log_2 p(x)$
- 信息增益: $IG = H(Y) - H(Y|X)$
- 增益率: $GainRatio = Gain / SplitInfo$
- Gini: $Gini = 1 - \sum p_k^2$

神经网络

- Sigmoid: $\sigma(x) = 1/(1+e^{-x})$, $\sigma'(x) = \sigma(x)(1-\sigma(x))$
- ReLU: $f(x) = \max(0, x)$, $f'(x) = \{0, x < 0; 1, x > 0\}$
- 前向传播: $y = \sigma(Wx + b)$
- 反向传播: $\partial L / \partial w = \partial L / \partial o \cdot \partial o / \partial z \cdot \partial z / \partial w$

Transformer

- Self-Attention: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$
- $Q = XW^Q$, $K = XW^K$, $V = XW^V$
- Multi-Head: $\text{MultiHead} = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^O$
- 残差连接: $X^{(l)} = \text{Layer}(X^{(l-1)}) + X^{(l-1)}$

强化学习

- 回报: $G_t = \sum \gamma^k R_{t+k+1}$
- $V(s): V\pi(s) = E\pi[G_t | S_t=s]$
- Bellman 最优: $V(s) = \max_a [R + \gamma \sum P \cdot V(s')]$
- TD 更新: $V(s) \leftarrow V(s) + \alpha[R + \gamma V(s') - V(s)]$
- Q-Learning: $Q(s, a) \leftarrow Q(s, a) + \alpha[R + \gamma \max_{a'} Q(s', a') - Q(s, a)]$

考试提示: 重点掌握各章标记 ★ 的内容，尤其是算法核心思想、关键公式和典型计算例题。

复习建议:

1. 逻辑推理（第二讲+第三四讲）：掌握命题/FOL 基础语法语义，重点练习归结证明和合一计算
2. 搜索技术：对比各算法性质，掌握 A* 最优性条件
3. Alpha-Beta 剪枝：亲手走一遍博弈树
4. 决策树：练习信息增益/Gini 计算
5. 神经网络：理解反向传播链式法则
6. Transformer：掌握 Self-Attention 公式和 QKV 计算流程
7. 强化学习：理解 MDP、Bellman 方程、SARSA vs Q-Learning